

Contents

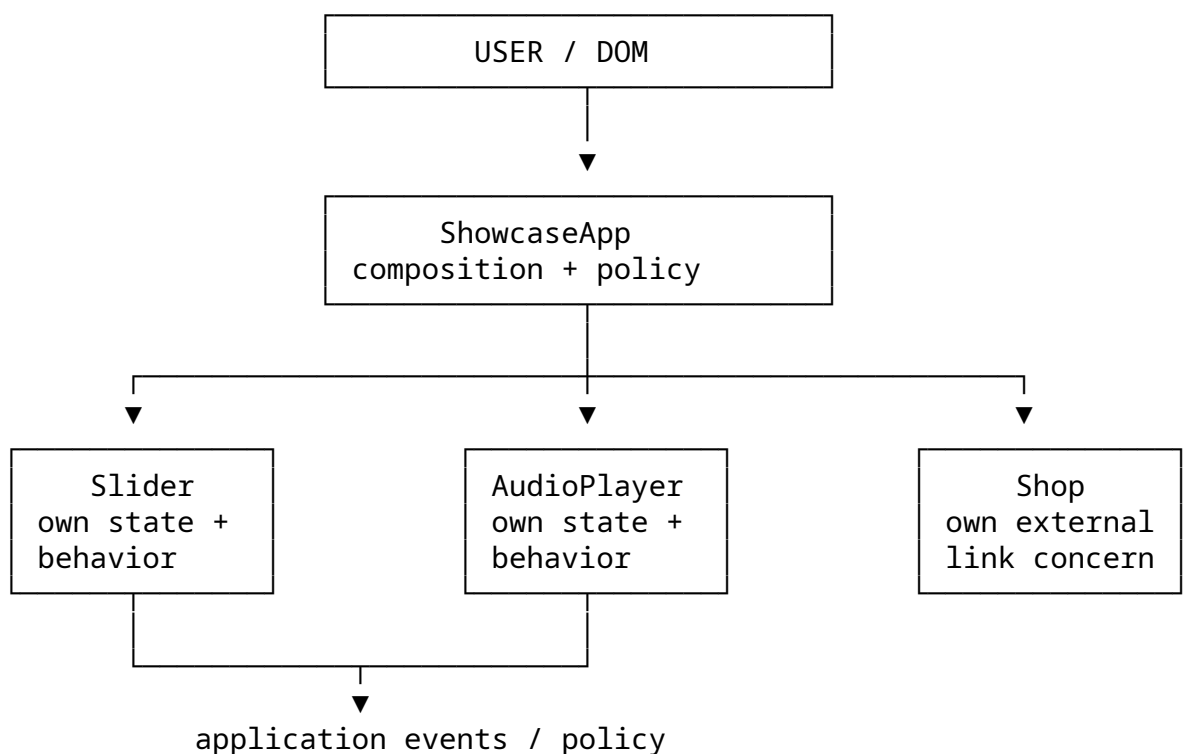
Обзор архитектуры	2
1. Как читать систему как единую идею	2
2. Философия проектирования	2
3. Архитектурные слои	3
4. Система с первого взгляда	3
5. Основные архитектурные принципы	4
5.1 Изоляция компонентов	4
5.2 Разделение ответственности	5
5.3 Конфигурация вместо дублирования	5
5.4 Fail-fast контракты	5
6. Композиция приложения	6
7. Компоненты не владеют друг другом напрямую	6
8. Архитектура Slider	7
9. Локальные конечные автоматы	8
10. FSM Slider	9
11. FSM Autoscroll	9
12. FSM AudioPlayer	9
13. Input Pipeline	10
14. Keyboard Pipeline	11
15. Менеджеры	11
16. EventManager	12
16.1 Declarative static maps	12
16.2 Dynamic subscriptions	12
17. KeyboardManager и ButtonManager	13
18. Button	13
19. Другая инфраструктура	13
19.1 Timer	13
19.2 DOMValidator	14
19.3 AudioDeckView	14
20. Карта архитектурных паттернов	14
21. Почему здесь нет глобального State Pattern	15
22. Взаимодействие архитектурных механизмов	16
23. Архитектурная философия	17
24. Финальная архитектура	17
24.1 Summary	18

Обзор архитектуры

1. Как читать систему как единую идею

На первый взгляд проект выглядит как интерактивный Slider с аудиоплеером. Функционально это верное описание, но с архитектурной точки зрения оно неполно. Самое интересное начинается в тот момент, когда несколько разных областей взаимодействия должны сосуществовать, не превращая приложение в один огромный объект.

Поэтому архитектура начинается не с набора функций, а с вопроса о владении ответственностью. Slider владеет движением. AudioPlayer владеет воспроизведением. Shop владеет своей задачей внешней ссылки. Переиспользуемые механизмы - подписка на события, диспетчеризация клавиатуры, обработка кнопок и работа с временем - вынесены в менеджеры. Над ними находится ShowcaseApp, который координирует последствия, пересекающие границы компонентов.



Такой взгляд объясняет, почему проект имеет больше структуры, чем обычный Slider. Каждый дополнительный слой появился потому, что возникла отдельная архитектурная ответственность, которой потребовалось собственное место.

2. Философия проектирования

Центральная философия проекта - **сложно, но контролируемо**, а не **минимально, но жёстко связано**. Это архитектурное учебное упражнение, поэтому проект намеренно исследует, что происходит, когда UI разложен на взаимодействующие объекты без использования фреймворка.

Цель не в абстракции ради самой абстракции. Полезна та абстракция, которая даёт концепции ясного владельца, устраняет дублирование механизма или создаёт переиспользуемый контракт. Поэтому система использует менеджеры, локальные FSM и pipeline, но намеренно не вводит принудительный глобальный State layer или универсальную обёртку вокруг каждой константы.

A real concern appears

Give it a clear owner

— domain concern -> component
— reusable mechanism -> manager

Connect across boundaries through application policy/events

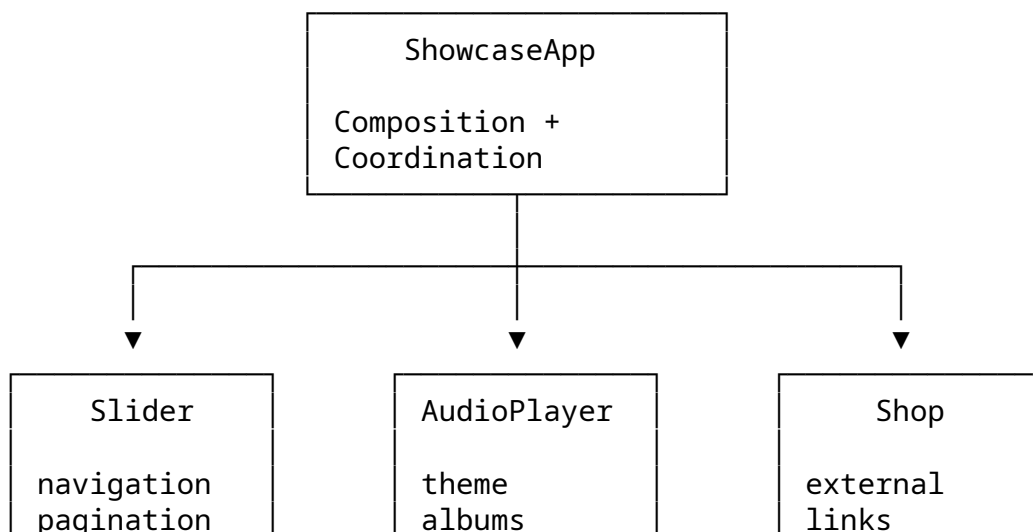
3. Архитектурные слои

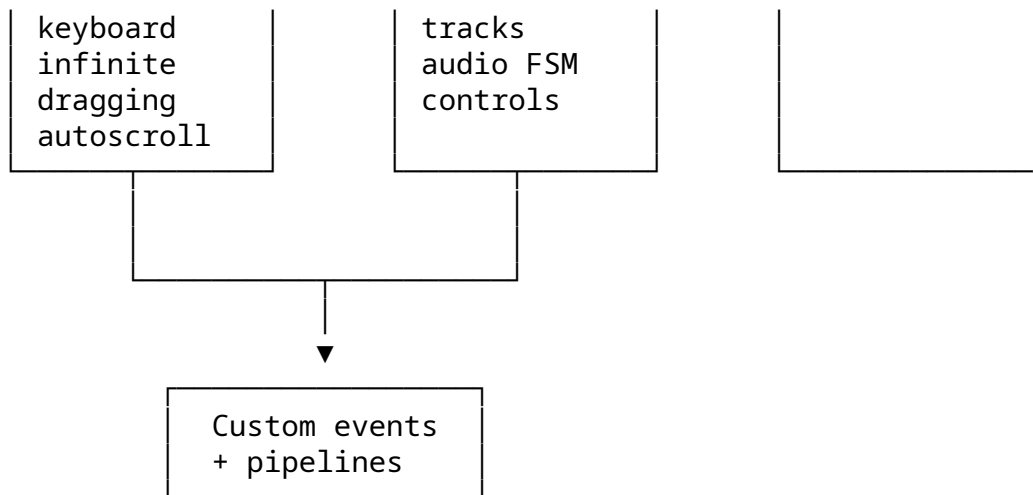
Всё приложение можно рассматривать как пять взаимодействующих слоёв:

1. Оркестрация приложения ShowcaseApp
2. Доменные компоненты Slider / AudioPlayer / AudioDeckView / Shop
3. Инфраструктура взаимодействия EventManager / KeyboardManager / ButtonManager Timer
4. Контракты / UI-поддержка DOMValidator / Button
5. Внутренние механизмы inheritance / polymorphism / FSM / _pipe

Остальная часть этого документа объясняет, зачем нужна такая декомпозиция и как её части взаимодействуют на высоком уровне.

4. Система с первого взгляда





Главное правило выглядит просто:

Компонент владеет своей доменной логикой

+

Менеджеры владеют переиспользуемыми механизмами взаимодействия

+

ShowcaseApp координирует взаимодействие компонентов

+

События и pipelines соединяют части

5. Основные архитектурные принципы

Изоляция компонентов

Каждый крупный компонент владеет собственным состоянием и поведением.

Slider

- └─ navigation
- └─ pagination
- └─ dragging
- └─ autoscroll
- └─ slider/autoscroll state

AudioPlayer

- └─ playback
- └─ theme
- └─ album/track selection
- └─ audio state

Shop

- └─ external link state

ShowcaseApp координирует эти компоненты, но не проникает во внутреннюю реализацию каждого из них.

Разделение ответственности

Ответственности намеренно распределены между:

Application

└─ ShowcaseApp

Components

├─ Slider
├─ AudioPlayer
├─ AudioDeckView
└─ Shop

Reusable infrastructure

├─ EventManager
├─ KeyboardManager
├─ ButtonManager
└─ Timer

Validation / UI abstractions

├─ DOMValidator
└─ Button

Utilities

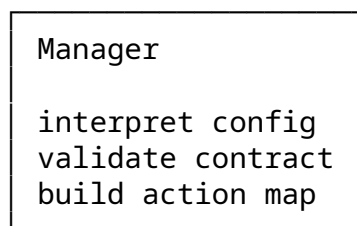
└─ helpers

Именно поэтому кода больше, чем в минимальной реализации Slider: проект исследует архитектуру, а не только UI-механику.

Конфигурация вместо дублирования

Правила взаимодействия там, где это полезно, задаются через configuration tables.

configuration



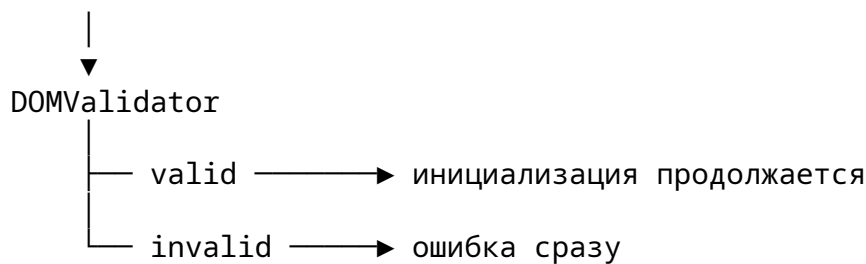
Component

Особенно хорошо это видно в keyboard, click и event handling.

Fail-fast контракты

Проект предпочитает явную ошибку защитным попыткам угадать, что имелось в виду.

Expected DOM

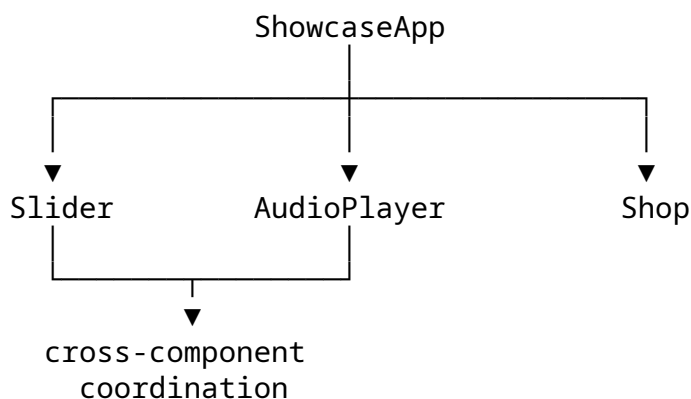


Если обязательная DOM-зависимость нарушена, архитектура предпочитает обнаружить это рядом с источником проблемы, а не маскировать её запасными селекторами.

6. Композиция приложения

ShowcaseApp является composition root.

Он получает основные компоненты и отвечает за их инициализацию, wiring и application-level coordination.



Его ответственности находятся на уровне приложения:

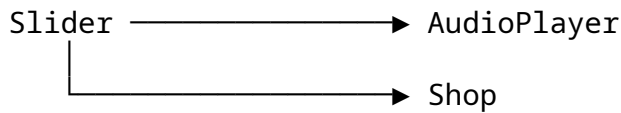
- создание и инициализация
- регистрация event maps
- маршрутизация общего browser input
- реакция на component events
- синхронизация Slider и AudioPlayer
- координация визуальных режимов

Он не реализует сам механизм перемещения Slider или механизм воспроизведения AudioPlayer.

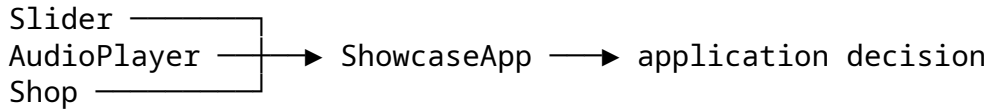
Это и есть практическое применение Inversion of Control и Dependency Injection: composition выполняется на границе приложения, а не внутри доменных компонентов.

7. Компоненты не владеют друг другом напрямую

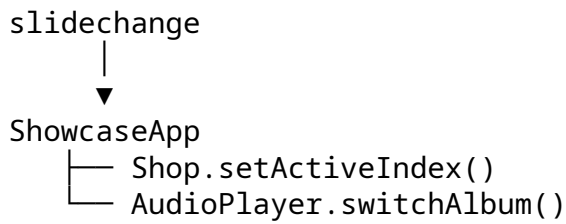
Нежелательная зависимость выглядела бы так:



Предпочтительный вариант:



Например:



Slider сообщает только:

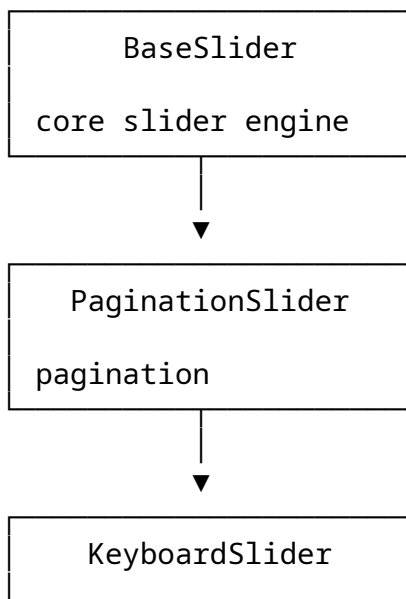
"активный slide изменился"

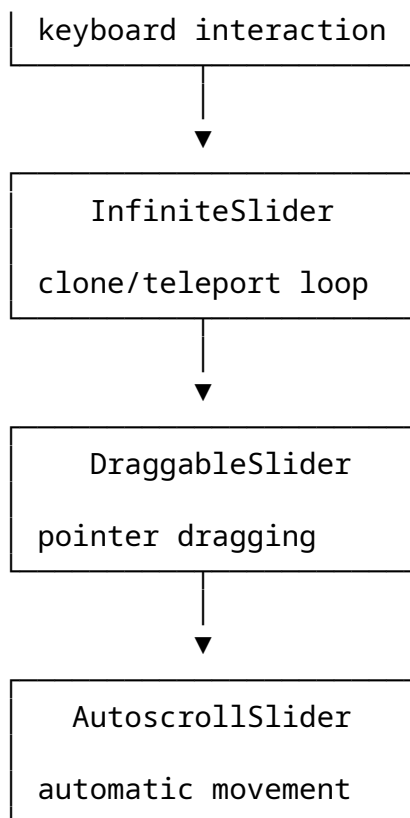
Ему не нужно знать, что такое album или purchase link.

Поэтому ShowcaseApp можно описывать как **Mediator-like координатор уровня приложения**. Его задача - держать cross-сomponent правила в одном месте, не перетаскивая внутрь себя доменную логику компонентов.

8. Архитектура Slider

Slider строится последовательно через inheritance.

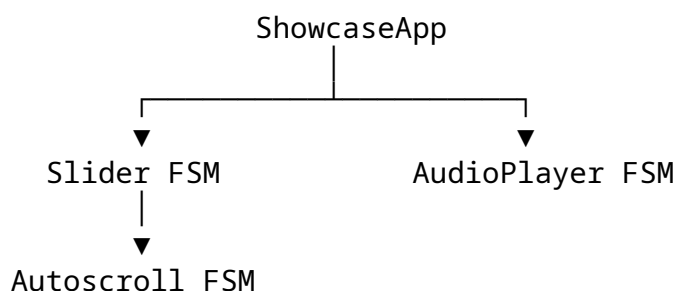




Каждый уровень добавляет одну возможность или специализирует существующий алгоритм. Это пример **capability-based extension через inheritance и polymorphism**, а там, где базовый класс задаёт общий алгоритм с точками расширения, структура также носит Template Method-like характер.

9. Локальные конечные автоматы

Проект сознательно не использует одну глобальную state machine.

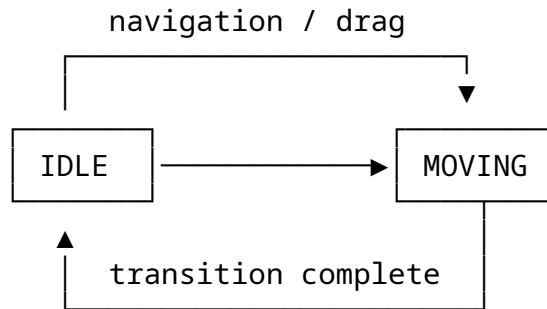


Есть три осмысленных state domains:

1. движение Slider
2. управление autoscroll
3. режим воспроизведения AudioPlayer

Компонент, который владеет состоянием, владеет и его смыслом и переходами.

10. FSM Slider

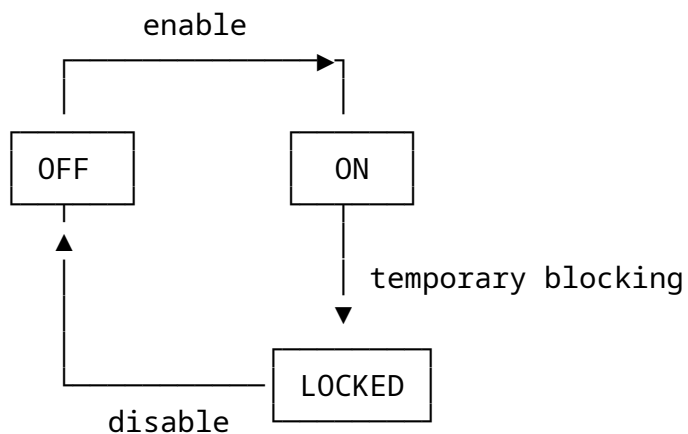


Главное различие:

`MOVING` $\neq$ `slidechange`

`MOVING` означает процесс движения, тогда как `slidechange` означает, что активный `slide` уже считается изменённым с точки зрения приложения.

11. FSM Autoscroll



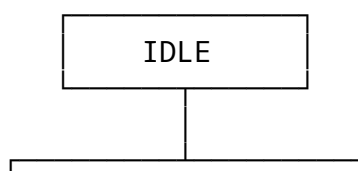
У **OFF** и **LOCKED** разный смысл:

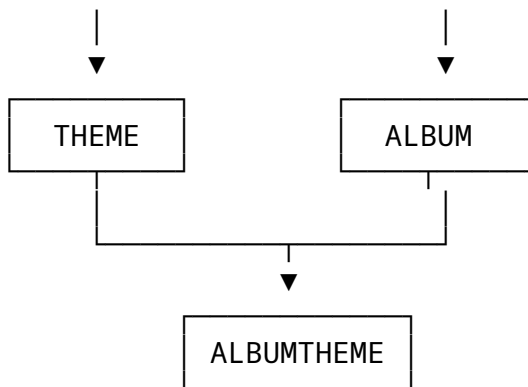
OFF = autoscroll отключён

LOCKED = autoscroll существует, но временно не может работать

Это особенно важно во время `album playback`.

12. FSM AudioPlayer

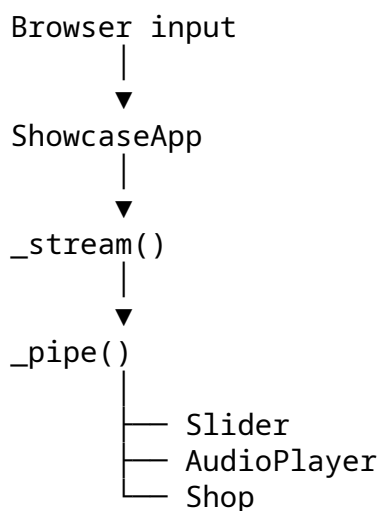




Здесь state означает не просто native <audio> status, а семантический playback context приложения.

13. Input Pipeline

Одна из характерных архитектурных идей проекта - `_pipe` interaction model.



Порядок может меняться.

NORMAL

Slider -> AudioPlayer -> Shop

REVERSE

AudioPlayer -> Slider -> Shop

Return value является небольшим control protocol:

true

└─ событие поглощено -> stop

original Event

└─ событие не обработано -> continue

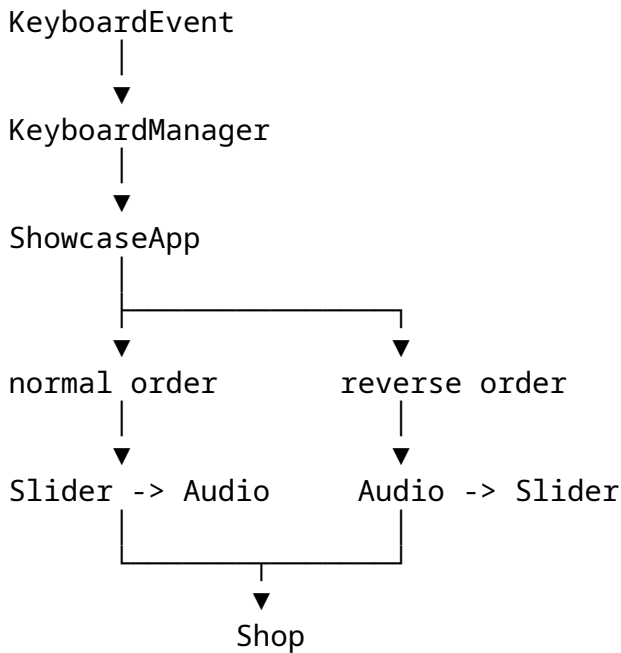
other result

└─ application/component-specific meaning

Поэтому pipeline можно рассматривать как лёгкую форму **Chain of Responsibility**.

14. Keyboard Pipeline

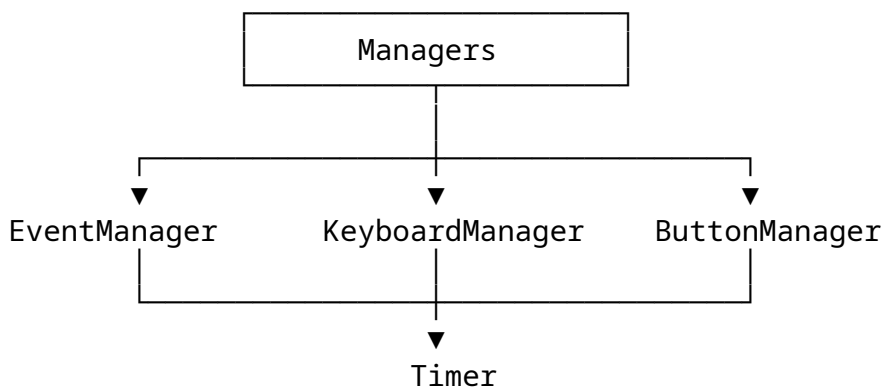
KeyboardManager классифицирует key event, ShowcaseApp определяет application-level routing, а компонент выполняет доменную операцию.



Reverse-порядок нужен для специальных ситуаций, в которых `AudioPlayer` должен получить возможность обработать ввод раньше `Slider`.

15. Менеджеры

Менеджеры являются центральным инфраструктурным слоем.



Главный принцип:

Manager = переиспользуемый механизм
Component = доменный смысл

Именно поэтому KeyboardManager не должен знать, что такое album, а EventManager - что такое dragging.

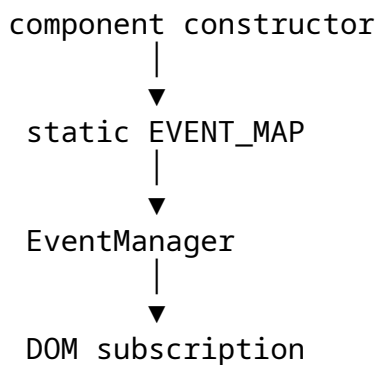
16. EventManager

EventManager предоставляет декларативную event routing infrastructure.

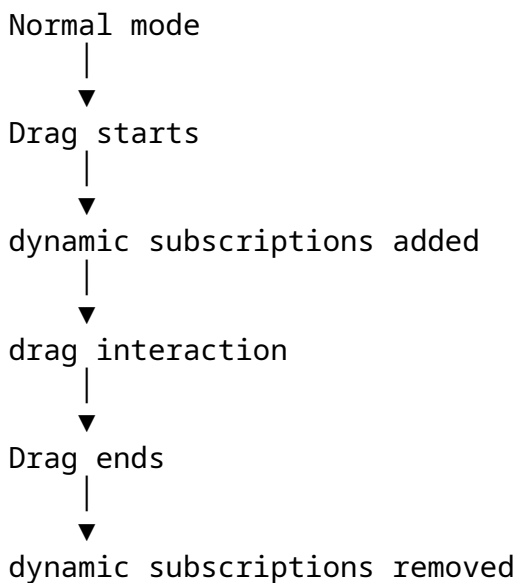
Он поддерживает:

- static event maps
- dynamic subscriptions
- subscription / unsubscription
- handler dispatch
- target resolution
- contract validation

Declarative static maps



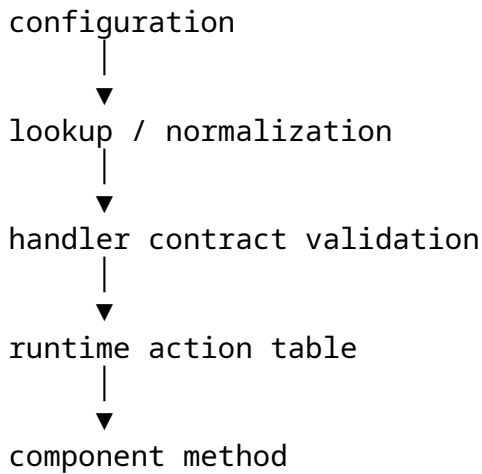
Dynamic subscriptions



Так EventManager становится местом, где сосредоточена инфраструктурная логика событий, а не доменная логика конкретного компонента.

17. KeyboardManager и ButtonManager

Оба менеджера реализуют близкую архитектурную идею:

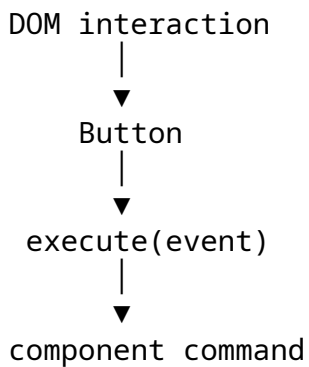


KeyboardManager работает с keyboard configuration, ButtonManager - с button/action mapping.

Компонент владеет смыслом команды; менеджер владеет механизмом её поиска и вызова.

18. Button

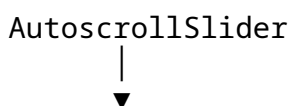
Button представляет UI-action в унифицированной форме.

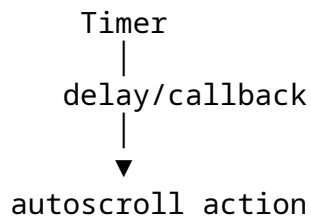


Это корректнее называть **Command-like abstraction**, а не пытаться выдавать его за textbook Command implementation.

19. Другая инфраструктура

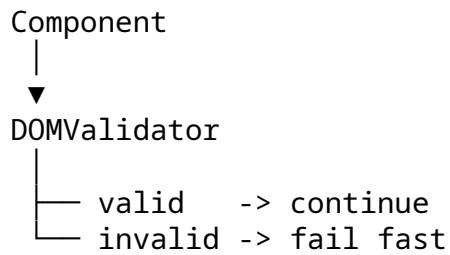
Timer





Timer не знает, что такое slide. Он владеет timing mechanism.

DOMValidator



AudioDeckView



View отображает информацию, но не владеет playback state.

20. Карта архитектурных паттернов

Проект использует несколько узнаваемых паттернов и принципов, но не все они являются textbook GoF implementations.

Паттерн / принцип	Где	Назначение
Mediator-like	ShowcaseApp	координация независимых компонентов
Observer / Pub-Sub-like	custom events + EventManager	событийная связь без прямого знания подписчиков
Chain of Responsibility	_pipe / propagation	обработчики могут поглотить или передать ввод дальше
Strategy-like	configurable managers	действие выбирается через configuration
Template Method-like	slider inheritance	общий алгоритм + полиморфные точки расширения
Command-like	Button	представление исполняемого UI-действия
FSM	Slider / Autoscroll / Audio-Player	локализация состояния и переходов
Composition	ShowcaseApp	сборка системы из независимых частей
Dependency Injection	application construction	внешняя передача зависимостей
Inversion of Control	composition root	разделение composition и domain logic
Polymorphism	slider hierarchy / static contracts	специализация поведения без дублирования

Важно различать названия:

DI = архитектурная техника

IoC = более широкий архитектурный принцип

FSM = техника моделирования состояния

Pub/Sub = стиль событийной коммуникации

Factory = использовать только там, где действительно присутствует соответствующая

21. Почему здесь нет глобального State Pattern

У компонентов уже есть ясные владельцы состояния:

```
BaseSlider
└─ IDLE / MOVING
```

```
AutoscrollSlider
└─ OFF / ON / LOCKED
```

```
AudioPlayer
└─ IDLE / ALBUM / THEME / ALBUMTHEME
```

Глобальный State object стал бы вторым владельцем той же информации и увеличил

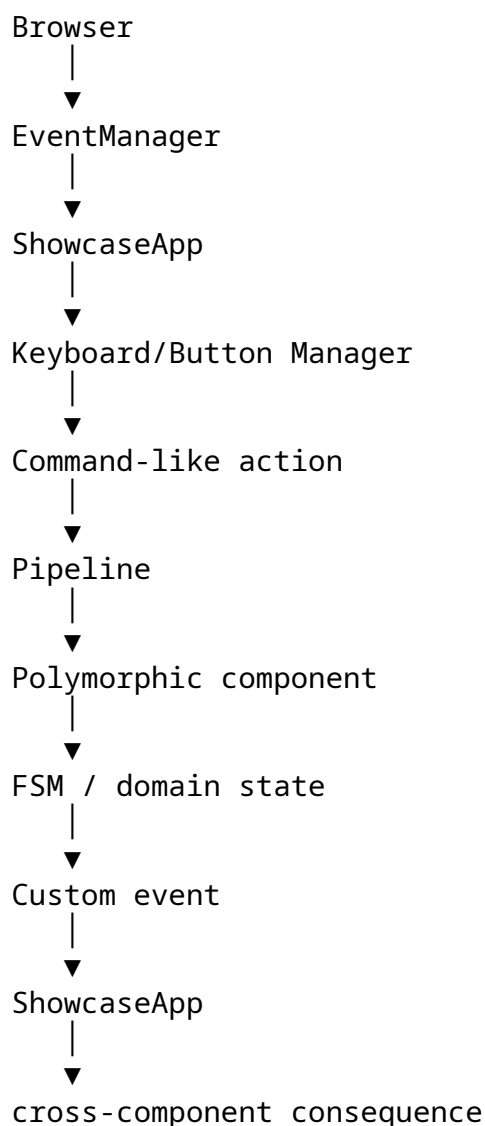
бы coupling.

local state ownership
 >
global state registry

Отдельная иерархия State objects могла бы стать полезной в будущем, если state-specific behavior вырастет настолько, что локальная модель перестанет быть удобной.

22. Взаимодействие архитектурных механизмов

Самое интересное в архитектуре - не количество паттернов, а то, как они соединяются.



Один пользовательский ввод может пройти через несколько архитектурных механизмов, при этом каждый сохраняет узкую ответственность.

23. Архитектурная философия

Проект сознательно выбирает:

complex but controllable
over
minimal but tightly coupled

Дополнительные абстракции существуют ради исследования:

границ компонентов
полиморфизма
наследования
FSM
переиспользуемых менеджеров
event routing
configuration
validation
application mediation

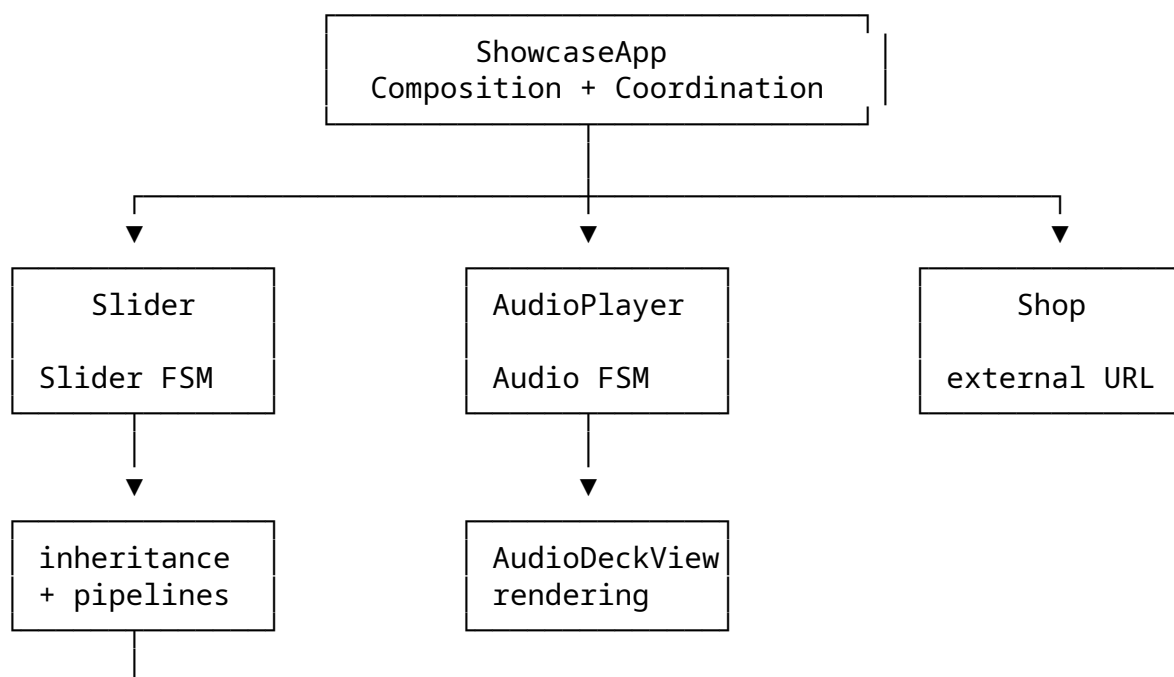
Поэтому архитектуру полезнее оценивать не вопросом:

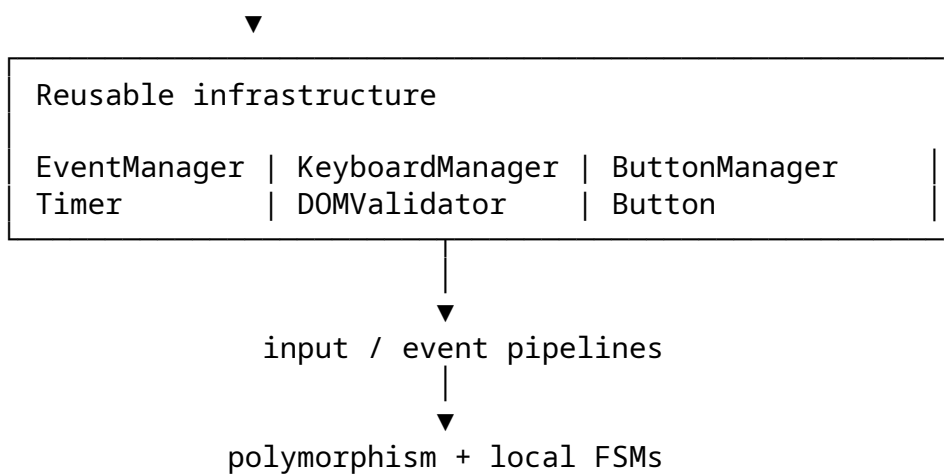
"Можно ли было написать меньше?"

а вопросом:

"Создаёт ли эта дополнительная структура
реальную ответственность, границу или механизм?"

24. Финальная архитектура





Summary

Архитектура складывается из пяти взаимодействующих concerns:

1. Оркестрация приложения
 - └─ ShowcaseApp
2. Доменные компоненты
 - └─ Slider
 - └─ AudioPlayer
 - └─ AudioDeckView
 - └─ Shop
3. Переиспользуемая инфраструктура взаимодействия
 - └─ EventManager
 - └─ KeyboardManager
 - └─ ButtonManager
 - └─ Timer
4. Валидация / UI abstractions
 - └─ DOMValidator
 - └─ Button
5. Внутренняя архитектура компонентов
 - └─ FSMs
 - └─ inheritance
 - └─ polymorphism
 - └─ input pipelines

Центральный архитектурный принцип:

Компоненты владеют поведением и состоянием.
 Менеджеры владеют переиспользуемыми механизмами.
 ShowcaseApp владеет композицией и межкомпонентной координацией.
 События и pipelines соединяют слои,
 не раскрывая лишние детали реализации.